Entender quien escribe mal: robustez de clasificadores de intenciones en español frente a ruido de escritura

Grupo 1 — Benitez, Tomás; Lombardia, Maximiliano;
Mónaco, Vianca; Vollert, Marcos

Universidad de Palermo — Procesamiento de Lenguaje Natural — 4/11/2026

Resumen

Los asistentes virtuales se entrenan con frases bien escritas, pero en producción reciben texto sin tildes, con errores de tipeo, abreviado o con expresiones regionales. Este trabajo estudia cuánto se degradan cinco clasificadores de intenciones en español ante ese ruido, y si las representaciones basadas en subpalabras resisten mejor que las basadas en palabras completas. Sobre el corpus MASSIVE (es-ES, 60 intenciones) se compararon TF-IDF de palabras y de caracteres con regresión logística, FastText con y sin subpalabras, y BETO ajustado por fine-tuning, frente a siete perturbaciones sintéticas en cinco intensidades. Con errores de tipeo la hipótesis se confirma: el mismo modelo FastText se degrada mucho menos cuando usa subpalabras. Sin embargo, BETO, que también usa subpalabras, es el mejor modelo sin ruido pero cae casi tanto como los modelos de palabras con typos intensos, mientras que TF-IDF de caracteres es el más robusto. Ante cambios de vocabulario y voseo, en cambio, BETO es el que menos se degrada. Como complemento, dos LLM locales usados sin entrenamiento no superaron a TF-IDF de caracteres en ninguna condición y fueron mucho más lentos.

1. Introducción

Un clasificador de intenciones decide qué quiere hacer el usuario de un asistente virtual (poner una alarma, consultar el clima, reproducir música) a partir de una frase. Los corpus con los que se entrenan estos sistemas suelen estar escritos con cuidado, pero los usuarios reales escriben rápido y de forma informal: omiten tildes, cometen errores de tipeo, abrevian o usan vocabulario y conjugaciones propias de su región. Un modelo con buen rendimiento en el conjunto de evaluación puede degradarse fuertemente ante esas variaciones.

La forma en que el modelo representa el texto debería influir en esa degradación. Un modelo que solo conoce palabras completas trata «alarma» y «slarma» como dos símbolos sin relación: el segundo nunca apareció en el entrenamiento y se ignora. Un modelo que mira fragmentos de la palabra («lar», «arm», «rma») todavía reconoce la mayoría de ellos en la forma mal escrita.

De ahí surge la hipótesis del trabajo:

> *Las representaciones que incorporan información de subpalabras (n-gramas de caracteres y tokenizadores de subpalabras de los transformers) son más resistentes a los errores de tipeo que las representaciones basadas exclusivamente en palabras completas.*

Como pregunta complementaria, se busca determinar en qué tipos de perturbación se manifiesta con mayor claridad esa diferencia. Los aportes del trabajo son:

- Un conjunto de siete perturbaciones controladas para español, con intensidad graduable, deterministas y anidadas entre niveles (§3.3).

- Una comparación de cinco clasificadores que incluye un *control experimental*: el mismo modelo FastText con y sin subpalabras, que aísla el efecto de la representación (§3.2).
- Una evaluación con intervalos de confianza por bootstrap pareado sobre frases, que distingue diferencias reales de variaciones de muestra (§3.4).
- Una comparación complementaria con dos LLM locales usados sin entrenamiento, que incluye latencia y consumo energético (§4.3).

2. Trabajos relacionados

Clasificación de texto. La clasificación de intenciones es un caso de clasificación de texto corto, una tarea que evolucionó desde los métodos tradicionales, con representaciones dispersas como TF-IDF sobre modelos de espacio vectorial y clasificadores lineales como la regresión logística, hasta los embeddings y los modelos profundos preentrenados. Este trabajo compara representantes de cada etapa de esa evolución sobre la misma tarea.

De palabras a subpalabras. En los modelos de espacio vectorial clásicos y en los embeddings estáticos como word2vec, cada palabra del vocabulario tiene su propia dimensión o su propio vector. Una palabra que no apareció en el entrenamiento, como una palabra con un error de tipeo, queda sin representación: es el problema de las palabras desconocidas. Una respuesta es descomponer las palabras en unidades menores. FastText, por ejemplo, representa cada palabra como la suma de los vectores de sus n-gramas de caracteres, y los tokenizadores de subpalabras dividen las palabras poco frecuentes en fragmentos de un vocabulario fijo.

Transformers y modelos preentrenados. La arquitectura transformer es la base de BERT, un codificador preentrenado sobre grandes corpus que luego se ajusta (*fine-tuning*) para cada tarea, como la clasificación de texto, con pocos datos etiquetados. BERT usa un tokenizador de subpalabras, por lo que, en principio, también debería beneficiarse de la descomposición de palabras desconocidas.

LLM como clasificadores. Los modelos de lenguaje grandes pueden resolver tareas sin ajustar sus pesos: sin ejemplos, a partir de una instrucción, o con unos pocos ejemplos en el prompt (*few-shot*). Esa capacidad mejora con la escala del modelo, pero también crecen su costo computacional y energético. Por eso tiene sentido preguntarse si un LLM chico, ejecutado localmente, es una alternativa competitiva a un clasificador entrenado.

Lo que aborda este trabajo. La bibliografía revisada explica por qué las subpalabras deberían ayudar ante palabras desconocidas, pero no cuantifica cuánto ayudan ante cada tipo de ruido de escritura ni si todos los esquemas de subpalabras se comportan igual. Este trabajo lo mide experimentalmente, en español y para clasificación de intenciones, con ruido de intensidad controlada.

3. Metodología

3.1. Datos

Se usó MASSIVE, un corpus de comprensión del lenguaje para asistentes virtuales publicado por Amazon Science, con alrededor de un millón de frases en 51 idiomas.[1] Se trabajó con la partición es-ES, que contiene frases cortas con 60 intenciones (por ejemplo `alarm_set`, `play_music`,

[1] `https://github.com/alexa/massive`

`weather_query`). Los tamaños son 11 514 frases de entrenamiento, 2 033 de validación y 2 974 de test. Los datos se cargaron desde la réplica `mteb/amazon_massive_intent` de Hugging Face, con una revisión fija, porque el repositorio original usa un script de carga que las versiones actuales de la biblioteca `datasets` ya no admiten. Se verificó que textos, etiquetas e identificadores coinciden con el parquet oficial en los tres conjuntos.

Uso de los conjuntos. Siguiendo la propuesta, **train** se usó para entrenar, **test** para elegir hiperparámetros y **validation** para la evaluación final, tanto sin ruido como con ruido. Es la convención inversa a la habitual, pero metodológicamente equivalente: los dos conjuntos provienen de la misma distribución y lo importante es que la robustez se mida sobre frases que no intervinieron en la selección de hiperparámetros. Por eso todo el rendimiento se reporta sobre validation.

Características relevantes. Las frases de MASSIVE ya vienen en minúsculas y casi sin puntuación (solo el 1,5 % tiene algún signo). Validation no contiene la intención `audio_volume_other`, por lo que tiene 59 intenciones, y seis de ellas tienen cinco frases o menos.

3.2. Modelos

Se entrenaron cinco clasificadores, agrupados según cómo representan el texto (Tabla 1). Los hiperparámetros de TF-IDF y FastText se eligieron con búsquedas pequeñas sobre test.

Modelo	Representación y clasificador	Grupo
TF-IDF palabras	TF-IDF de 1 y 2-gramas de palabras + regresión logística	Palabras
TF-IDF caracteres	TF-IDF de n-gramas de caracteres (2 a 5) dentro de cada palabra + regresión logística	Subpalabras
FastText sin subpalabras	FastText supervisado, solo palabras y bigramas	Palabras
FastText con subpalabras	FastText supervisado con n-gramas de caracteres de 3 a 6	Subpalabras
BETO	Fine-tuning de BERT preentrenado en español, con tokenizador WordPiece	Subpalabras

Tabla 1: Modelos comparados.

TF-IDF y regresión logística. Cada frase se representa como un vector disperso ponderado por TF-IDF, con frecuencia sublineal $(1 + \log \mathrm{tf})$. La variante de caracteres usa n-gramas que no cruzan los espacios entre palabras. Sobre ese vector se entrena una regresión logística multiclase con $C = 10$ (se probaron 1, 10 y 30). Se implementó con scikit-learn.

FastText. Aprende un embedding de 100 dimensiones por palabra, promedia los de la frase y clasifica con una capa lineal (`lr` = 0,5, 100 épocas, bigramas de palabras). Con subpalabras, el vector de cada palabra es la suma de los vectores de sus n-gramas de 3 a 6 caracteres, de modo que una palabra mal escrita recibe un vector cercano al de la original. La variante sin subpalabras usa exactamente los mismos hiperparámetros y es el control experimental del trabajo: si la versión con subpalabras resiste más, la diferencia se debe solo a la representación. Como FastText 0.9.2 no es determinista dentro de un mismo proceso, cada entrenamiento se hace en un proceso nuevo.

BETO. BETO es un modelo BERT preentrenado en español por la Universidad de Chile. Se partió de su versión `dccuchile/bert-base-spanish-wwm-cased`,[2] se le agregó una capa de

[2] `https://huggingface.co/dccuchile/bert-base-spanish-wwm-cased`

clasificación y se ajustó durante 3 épocas (tasa de aprendizaje 5×10^{-5}, lote de 32, *weight decay* de 0,01, *warmup* del 10 %, máximo de 64 tokens) con la biblioteca Hugging Face Transformers, siguiendo el procedimiento habitual de clasificación con transformers. Su tokenizador WordPiece deja las palabras frecuentes enteras y parte las raras o mal escritas en fragmentos: «alarma» es un único token, pero «slarma» se divide en «s», «lar» y «ma». Por eso cuenta como modelo de subpalabras, aunque con un vocabulario fijo.

Semillas de entrenamiento. FastText y BETO se entrenaron con tres semillas (42 a 44) y se promediaron. La regresión logística con el solver *lbfgs* produce el mismo modelo con cualquier semilla, así que TF-IDF se entrenó una sola vez.

3.3. Perturbaciones

Se diseñaron siete perturbaciones que modelan fenómenos frecuentes en la escritura informal (Tabla 2). Cada una tiene una intensidad $p \in [0, 1]$ y se evaluó en $p \in \{0{,}1;\ 0{,}25;\ 0{,}5;\ 0{,}75;\ 1{,}0\}$.

Perturbación	Qué hace	Cómo actúa p
Sin tildes	«canción» → «cancion» (conserva la ñ)	probabilidad por carácter acentuado
Minúsculas y sin puntuación	elimina mayúsculas y los signos `. , ¿ ? ¡ !`	probabilidad por frase
Errores de tipeo	tecla vecina en un teclado QWERTY español, letra borrada, duplicada o dos letras intercambiadas	probabilidad por palabra de 3 letras o más
Abreviaturas	«por favor» → «xfa», «que» → «q», «para» → «pa», «cinco» → «5» (32 entradas)	probabilidad por ocurrencia
Voseo rioplatense	«pon» → «poné», «dime» → «decime», «enciende» → «prendé» (16 entradas)	probabilidad por ocurrencia
Vocabulario coloquial	«correo electrónico» → «mail», «canción» → «tema», «necesito» → «me hace falta» (18 entradas)	probabilidad por ocurrencia
Combinado	todas en cadena: coloquial, voseo, abreviaturas, minúsculas, tildes y typos	la misma p para todas

Tabla 2: Perturbaciones aplicadas sobre validation.

Diseño. Cada unidad de la frase (palabra, carácter acentuado u ocurrencia de una entrada del diccionario) tiene su propio generador aleatorio, que depende de la semilla, de la perturbación, del texto y de la posición, pero no de p ni de las demás frases. Esto garantiza tres propiedades, verificadas con tests automáticos: el mismo conjunto se genera idéntico cada vez; una frase recibe el mismo ruido sola o dentro de un lote; y los niveles son *anidados*, es decir, lo que se altera con $p = 0{,}1$ sigue alterado igual con $p = 0{,}25$. Así, entre un nivel y el siguiente solo se agregan errores, y las curvas de degradación no mezclan el efecto de la intensidad con el azar de qué palabras se eligieron. En los typos, primero se elige la operación, de forma equiprobable, y luego una posición donde esa operación efectivamente modifique la palabra.

Diccionarios. Los diccionarios de abreviaturas, voseo y vocabulario coloquial se armaron a mano. Cada reemplazo se revisó sobre los contextos en que la palabra aparece en *train*, no en validation ni en test, para no diseñar el ruido mirando el conjunto de evaluación. Se descartaron reemplazos que cambiaban el significado en el corpus: por ejemplo, «vale» → «dale» (en MASSIVE es el verbo *valer*), «móvil» → «celu» (aparece como adjetivo en «media móvil») o «uno» → «1» (casi siempre es pronombre). El voseo y el vocabulario se separaron porque son fenómenos distintos: el primero cambia la morfología del verbo y conserva gran parte de la forma escrita, mientras que el segundo reemplaza una palabra por otra sin relación ortográfica. En la

perturbación combinada los diccionarios se aplican antes que los typos, porque buscan palabras bien escritas.

Semillas de ruido. Cada perturbación se generó con cinco semillas (42 a 46). En total se evaluaron 176 conjuntos de validation: el limpio más 7 perturbaciones × 5 intensidades × 5 semillas. La Tabla 3 muestra ejemplos reales.

Perturbación (p)	Frase
Original	envía un correo electrónico a mi jefe diciendo que llego tarde
Sin tildes (1,0)	envia un correo electronico a mi jefe diciendo que llego tarde
Errores de tipeo (1,0)	ejvía un corrreo electróinco a mi jfe diciedno ...
Voseo (1,0)	mandá un correo electrónico a mi jefe diciendo ...
Coloquial (1,0)	envía un mail a mi jefe diciendo que llego tarde
Combinado (1,0)	mqnda un maiil a mi jfee diciedno q lleggo tarse
Original	ponme una alarma para mañana a las siete por favor
Abreviaturas (0,5)	ponme una alarma pa mñn a las 7 xfa
Voseo (0,5)	poneme una alarma para mañana a las siete por ...

Tabla 3: Ejemplos de perturbaciones generadas.

3.4. Evaluación

Métricas. Se reportan accuracy y F1 macro. El F1 macro promedia el F1 de cada intención sin ponderar por frecuencia, de modo que penaliza fallar en intenciones poco frecuentes. Se promedia solo sobre las intenciones presentes en el conjunto (59 en validation): si se incluyeran las intenciones predichas que no están en el conjunto, cada una sumaría un F1 de 0 y el promedio dejaría de ser comparable entre modelos y niveles de ruido.

Caída de rendimiento. Los modelos no rinden igual sin ruido, por lo que la robustez se compara mediante la *caída*: la accuracy limpia menos la accuracy con ruido, en puntos absolutos, calculada sobre las mismas frases. Esto también es necesario porque sin ruido los modelos ya se equivocan en el 13 al 21 % de las frases: con typos a $p = 0{,}25$, entre el 52 y el 75 % de los errores ya existían en la versión limpia.

Intervalos de confianza. La caída se acompaña de un intervalo de confianza del 95 % obtenido por *bootstrap pareado*: se remuestrean frases con reposición (1 000 remuestreos) y cada frase aporta juntas su versión limpia y sus versiones con ruido, promediadas sobre semillas. Para comparar dos modelos se calcula, del mismo modo, la diferencia entre sus caídas; si el intervalo no incluye el 0, la diferencia no se explica por la muestra de frases. Se usa accuracy porque se descompone por frase, lo que no ocurre con el F1 macro. Estos intervalos no incluyen la variabilidad entre semillas de entrenamiento, que se informa por separado.

3.5. Comparación complementaria con LLM locales

Se agregó una comparación opcional con LLM chicos ejecutados localmente mediante Ollama:[3] `llama3.1` (8B parámetros, de Meta) y `qwen3.5:4b` (4B parámetros, de Alibaba). No se entrenan: el prompt de sistema contiene las 60 etiquetas y, con $k = 2$, dos ejemplos de train por intención, al estilo *few-shot*. La respuesta se restringe con un esquema JSON a una de las 60 etiquetas, con temperatura 0 y un contexto de 4 096 tokens. En `qwen3.5:4b` se desactivó

[3] `https://ollama.com`

el modo de razonamiento, porque de lo contrario agota los tokens permitidos y devuelve una respuesta vacía.

El valor de k se eligió con 500 frases limpias de test, con el mismo criterio que los hiperparámetros de los demás modelos. `llama3.1` obtuvo 0,572 con $k = 0$ y 0,630 con $k = 2$; `qwen3.5:4b`, 0,696 y 0,702. Por su costo, los LLM se evaluaron sobre validation solo con intensidades 0,25 y 1,0 y con una única semilla de ruido. También se midió la latencia por frase, el rendimiento en lote y la energía consumida por la GPU.

3.6. Implementación y reproducibilidad

El código está en Python. Cada modelo entrenado se guarda con un nombre que incluye un *hash* de su configuración y de la revisión de los datos, de modo que un cambio de hiperparámetros obliga a reentrenar en lugar de reutilizar un modelo viejo. Cada ejecución registra el commit del código, las versiones de las bibliotecas y la plataforma. Los resultados principales se obtuvieron en una GPU NVIDIA RTX 3080; la corrida completa de los cinco modelos con todas sus semillas tarda alrededor de cinco minutos.

4. Experimentos y resultados

4.1. Rendimiento sin ruido

La Tabla 4 y la Figura 1 muestran el rendimiento sobre validation sin ruido. BETO es el mejor en accuracy (0,872). En F1 macro, BETO (0,819) y TF-IDF de caracteres (0,807) quedan dentro de la variabilidad entre semillas de BETO (±0,014). En los dos pares de modelos comparables, la versión con subpalabras rinde mejor también sin ruido.

Modelo	Accuracy [IC 95 %]	F1 macro	Semillas
TF-IDF palabras	0,813 [0,794; 0,829]	0,780	1
TF-IDF caracteres	0,842 [0,826; 0,857]	0,807	1
FastText sin subpalabras	0,789 [0,772; 0,807]	0,766 ± 0,002	3
FastText con subpalabras	0,821 [0,805; 0,838]	0,795 ± 0,002	3
BETO	**0,872** [0,858; 0,885]	**0,819** ± 0,014	3

Tabla 4: Rendimiento sin ruido sobre validation. El ± es el desvío entre semillas de entrenamiento.

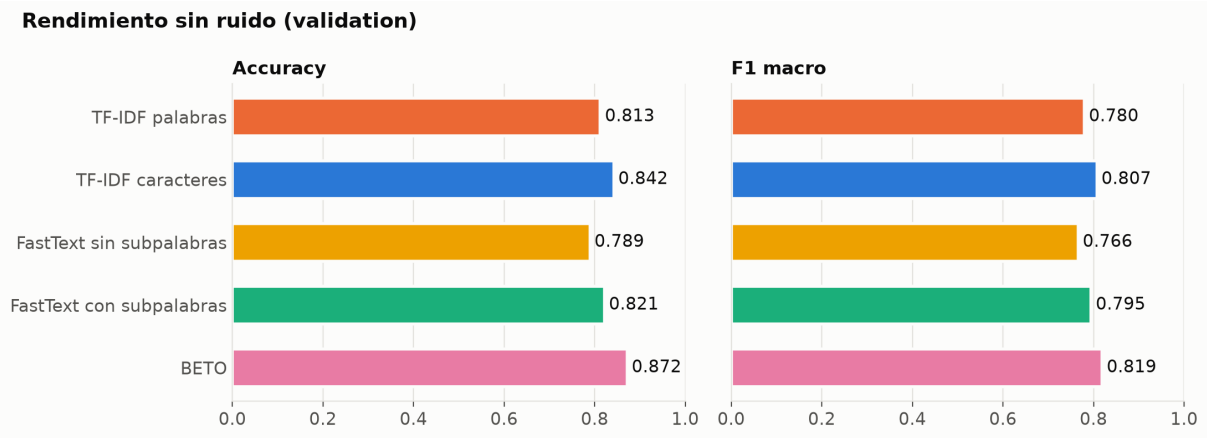


Figura 1: Accuracy y F1 macro sin ruido sobre validation.

4.2. Degradación frente al ruido

La Figura 2 muestra el F1 macro según la intensidad de cada perturbación, y la Tabla 5 la caída de accuracy a intensidad máxima. Las perturbaciones se separan en dos grupos muy distintos. Los errores de tipeo y el ruido combinado degradan fuertemente a todos los modelos: con typos a $p = 1{,}0$, TF-IDF de palabras pasa de un F1 macro de 0,780 a 0,074, y FastText sin subpalabras queda en un nivel similar. El resto de las perturbaciones produce caídas pequeñas, de 0,057 puntos o menos.

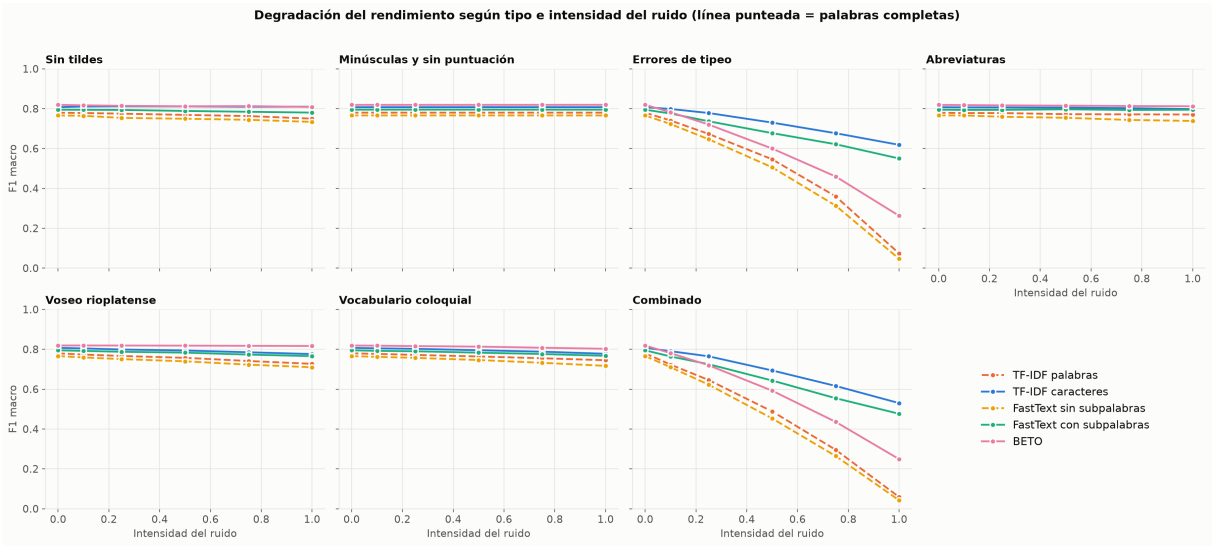


Figura 2: F1 macro según tipo e intensidad del ruido, promediado sobre semillas. Las líneas punteadas corresponden a modelos de palabras completas.

Ruido ($p = 1{,}0$)	TF-IDF pal.	TF-IDF car.	FastText sin sub.	FastText con sub.	BETO
Errores de tipeo	0,681	**0,195**	0,687	0,239	0,574
Combinado	0,691	**0,303**	0,682	0,333	0,599
Vocabulario coloquial	0,036	0,034	0,057	0,034	**0,011**
Sin tildes	0,033	0,015	0,031	0,020	0,014
Voseo	0,021	0,011	0,031	0,013	**0,003**
Abreviaturas	0,009	0,009	0,017	0,014	0,007
Minúsculas y sin puntuación	0	0	0	0	0

Tabla 5: Caída de accuracy en validation (limpia menos con ruido, en puntos absolutos) a intensidad 1,0. Menor es mejor.

Errores de tipeo: la hipótesis se confirma. La comparación más limpia es la de FastText con y sin subpalabras, que comparten algoritmo e hiperparámetros. La versión sin subpalabras cae 0,448 puntos más a intensidad 1,0 (IC 95 %: [0,426; 0,471]) y 0,067 más a intensidad 0,25 ([0,058; 0,078]). La diferencia es significativa en todas las intensidades de typos y de ruido combinado. TF-IDF de caracteres es el modelo más robusto, con una caída de 0,195 frente a 0,681 de TF-IDF de palabras.

BETO matiza la hipótesis. Aunque BETO usa subpalabras, con typos a intensidad 1,0 pierde 0,574 puntos: mucho más que TF-IDF de caracteres (diferencia de caídas de 0,378, IC [0,361; 0,395]), aunque menos que TF-IDF de palabras (0,107 menos, IC [0,084; 0,128]). A intensidad 0,25 no se distingue de TF-IDF de palabras. Los dos tipos de subpalabras difieren

en cuánto se solapan las representaciones de la palabra limpia y la mal escrita: en los typos a intensidad 1,0, el 81 % de los pares palabra–typo no comparte *ningún* token WordPiece con la palabra original, contra el 3,9 % que no comparte ningún n-grama de caracteres. Esto es consistente con que lo que importa para la robustez es ese solapamiento y no el hecho de usar subpalabras en sí, aunque se trata de una descripción y no de una prueba causal.

Qué ruido afecta a quién. La Figura 3 resume la caída relativa del F1 macro por modelo y perturbación, y la Figura 4 la promedia por tipo de representación. Separar BETO de los modelos de n-gramas de caracteres es necesario: si se los promedia juntos, la diferencia entre ambos tipos de subpalabras queda oculta.

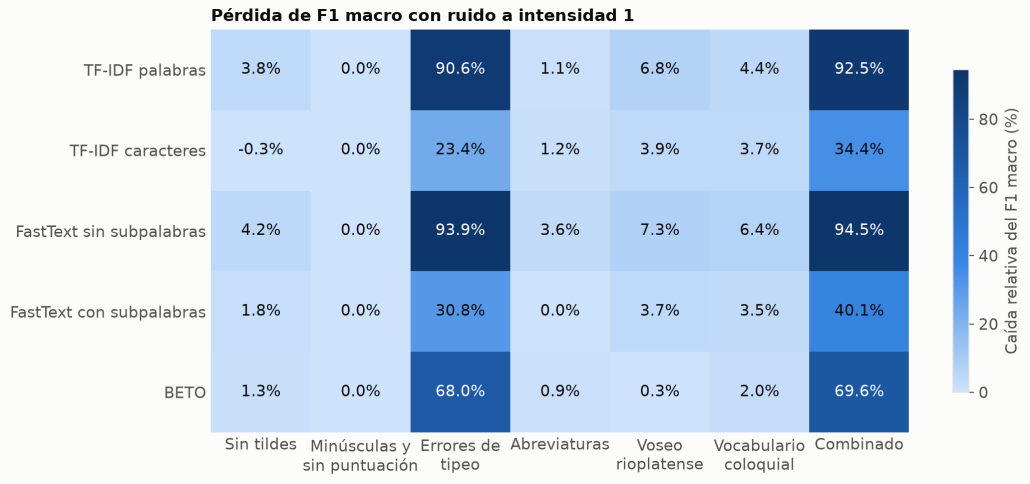


Figura 3: Caída relativa del F1 macro (%) a intensidad 1,0, por modelo y perturbación.

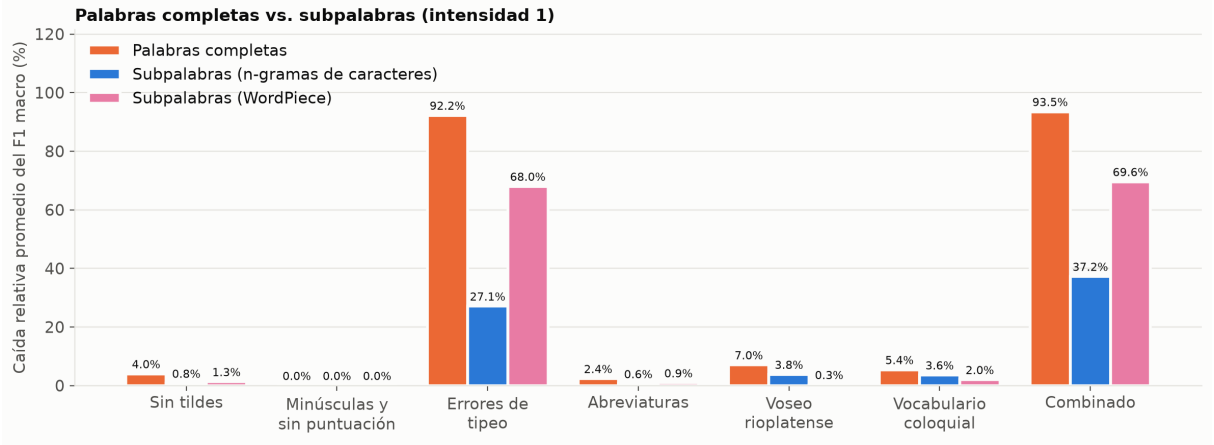


Figura 4: Caída relativa promedio del F1 macro a intensidad 1,0 por tipo de representación.

Vocabulario coloquial y voseo. En estos casos BETO es el más robusto (caídas de 0,011 y 0,003). En TF-IDF, palabras y caracteres no difieren ante el vocabulario coloquial (diferencia de 0,001, IC [−0,007; 0,009]): «mail» no comparte n-gramas con «correo», así que es un problema de vocabulario que las subpalabras no pueden compensar. En FastText la diferencia existe pero es chica (0,023, IC [0,013; 0,034]). Es plausible que el preentrenamiento de BETO sobre un corpus amplio incluya estas variantes, aunque no se verificó.

Tildes, abreviaturas, minúsculas. El efecto de quitar tildes es pequeño en todos los modelos ($\leq 0{,}033$) y no se puede afirmar que BETO supere a TF-IDF de caracteres (diferencia de 0,001, IC [−0,008; 0,011]). Las abreviaturas tienen un efecto de 0,017 o menos, sin diferencias significativas entre modelos. La perturbación de minúsculas y puntuación no tiene efecto, porque MASSIVE ya viene en minúsculas y casi sin puntuación.

4.3. Comparación con LLM locales

La Tabla 6 y la Figura 5 comparan los LLM con los modelos entrenados. Ningún LLM supera a TF-IDF de caracteres en ninguna condición: con typos a intensidad 1,0, el mejor LLM, `qwen3.5:4b`, obtiene 0,554 contra 0,647. Tampoco superan a FastText con subpalabras, salvo un empate con ruido combinado a intensidad 1,0. Contra BETO sí hay un cruce: BETO empieza muy por encima pero termina por debajo de ambos LLM con typos intensos (diferencia de caídas con `qwen3.5:4b` de 0,398, IC [0,373; 0,423]); el cruce se observa en la figura pero no se calculó con un intervalo de confianza.

En cuanto a la robustez, `qwen3.5:4b` con $k = 2$ cae ante typos casi lo mismo que TF-IDF de caracteres (diferencia de −0,020, IC [−0,042; 0,002]), mientras que con $k = 0$ cae más (0,049, IC [0,025; 0,074]). Sin ruido, $k = 0$ y $k = 2$ rinden casi igual (0,725 y 0,730): los ejemplos del prompt ayudan más a la robustez que al nivel base.

El costo es mucho mayor. La latencia mediana de los LLM es unas mil veces la de TF-IDF de caracteres y unas noventa veces la de BETO. Por cada mil frases, la GPU consume 0,017 Wh con BETO, 12,7 Wh con `llama3.1` y 21,0 Wh con `qwen3.5:4b`. TF-IDF y FastText corren en CPU, y la medición de la GPU no los representa. `qwen3.5:4b` resultó más lento y de mayor consumo que `llama3.1` pese a ser más chico; no se investigó la causa. Para esta tarea, entonces, el costo energético de los modelos grandes no se traduce en una mejora de robustez.

Modelo	Limpio	Typos 1,0	Combinado 1,0	p50 (ms)	Frases/s
TF-IDF caracteres	0,842	**0,647**	**0,539**	0,54	26 373
FastText con subpalabras	0,821	0,582	0,489	0,012	33 974
BETO	**0,872**	0,298	0,273	6,25	3 047
`llama3.1` ($k = 2$)	0,670	0,470	0,412	569	7,0
`qwen3.5:4b` ($k = 2$)	0,730	0,554	0,494	1 062	3,8

Tabla 6: Accuracy en validation y costo de inferencia. p50: latencia mediana al clasificar una frase por vez. Frases/s: rendimiento al clasificar el conjunto completo.

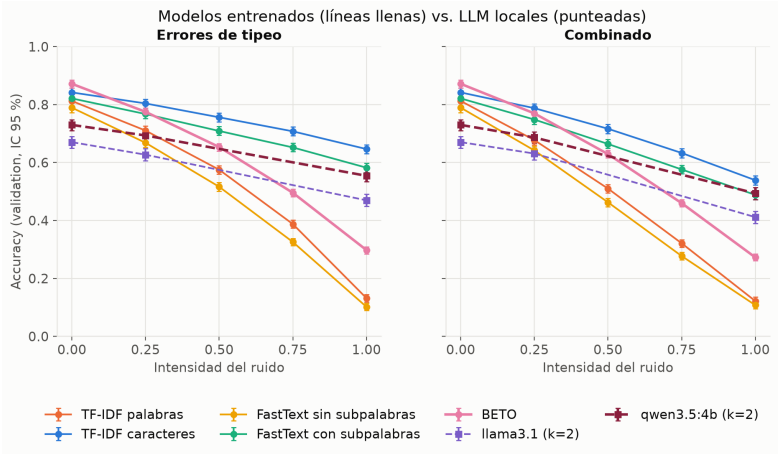


Figura 5: Accuracy (IC 95 %) según la intensidad de los typos y del ruido combinado. Los LLM solo se evaluaron a intensidad 0,25 y 1,0.

5. Discusión, análisis de errores y limitaciones

5.1. Análisis de errores

Confusión de base. Sin ruido, los errores se concentran en pares de intenciones semánticamente cercanas: en promedio entre modelos, el 33 % de las frases de `music_settings` se clasifica como `play_music`, el 24 % de `datetime_convert` como `datetime_query` y alrededor del 21 % de `qa_maths` como `qa_factoid`. La intención `general_quirky`, que agrupa frases variadas, funciona como sumidero de errores. Con ruido el fenómeno se acentúa: cuando TF-IDF de palabras no reconoce las palabras de la frase, la predicción tiende a `general_quirky`, que es la confusión más frecuente con ruido combinado a intensidad 0,5.

Dónde ganan los n-gramas de caracteres. Con typos a intensidad 0,5, hay 438 frases en las que TF-IDF de caracteres acierta y TF-IDF de palabras falla, contra 76 en el sentido contrario. Un ejemplo típico es «puedes crrear uha nueca listta» (`lists_createoradd`): las tres palabras con typo son desconocidas para el modelo de palabras, que predice `general_quirky`, mientras que los n-gramas de caracteres conservan suficientes fragmentos para acertar. En los casos inversos, el typo suele producir fragmentos que se parecen a otra intención: «avísame a las cino hora» se clasifica como `calendar_set` en lugar de `alarm_set`.

Intenciones más afectadas. La Figura 6 muestra las diez intenciones que más F1 pierde TF-IDF de palabras con ruido combinado a intensidad 0,5. En todas ellas TF-IDF de caracteres pierde bastante menos. Las más afectadas dependen de pocas palabras clave (`iot_wemo_on`, `audio_volume_down`, `alarm_remove`), y basta con alterar esas palabras para que el modelo de palabras se quede sin evidencia.

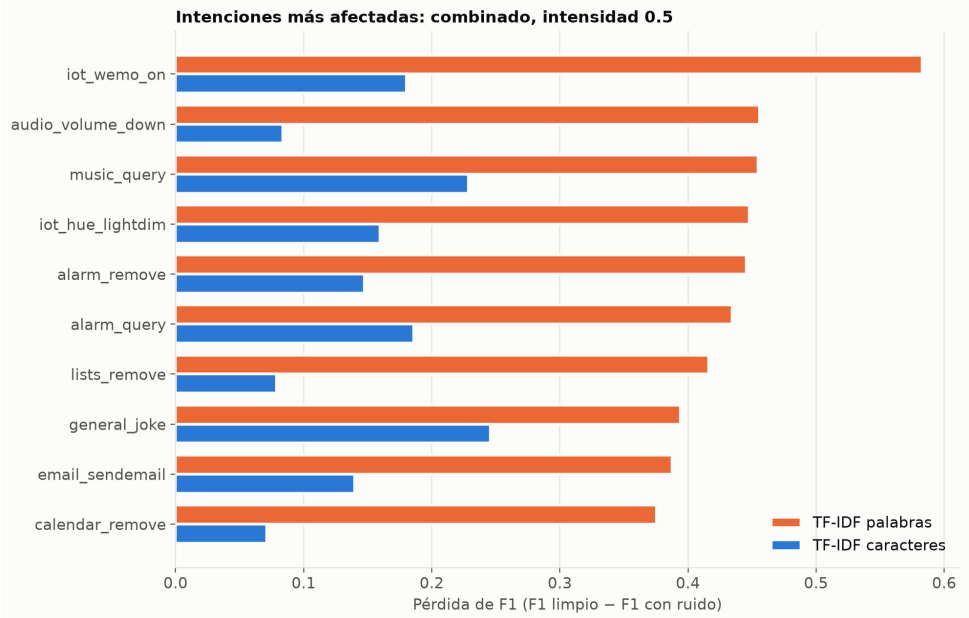


Figura 6: Intenciones con mayor pérdida de F1 para TF-IDF de palabras con ruido combinado a intensidad 0,5, comparadas con TF-IDF de caracteres.

Abreviaturas y modelos de palabras. De las abreviaturas usadas, solo «cumple» aparece en el vocabulario de entrenamiento. Un modelo TF-IDF ignora los tokens desconocidos, así que para TF-IDF de palabras abreviar equivale a borrar la palabra. Que la caída sea pequeña indica que las palabras abreviadas («de», «que», «por», «para») aportan poca información sobre la intención, no que el modelo entienda las abreviaturas.

5.2. Limitaciones

- **Ruido sintético.** Las perturbaciones aproximan, pero no reproducen, los errores reales de los usuarios. La intensidad 1,0 de typos (un error en cada palabra de tres o más letras) es extrema y sirve como cota más que como escenario realista.
- **Diccionarios hechos a mano.** Los diccionarios de abreviaturas, voseo y vocabulario coloquial son chicos, tienen sesgo rioplatense y se aplican sobre un corpus de español de España. Algunos reemplazos son aproximados («taxi» → «remis», «reunión» → «junta»).
- **Perturbación sin efecto.** Como MASSIVE ya viene en minúsculas y casi sin puntuación, la perturbación correspondiente no pudo evaluarse.
- **Tamaño de la evaluación.** La robustez se mide sobre 2 033 frases, y algunas intenciones tienen cinco frases o menos, por lo que su F1 es inestable.
- **Comparaciones múltiples.** Se calcularon 56 comparaciones con intervalos del 95 %; es esperable que unas tres excluyan el cero por azar. Las conclusiones principales se apoyan en diferencias muy por encima de ese umbral.
- **Variabilidad.** Los intervalos de confianza reflejan la muestra de frases, no la variabilidad entre semillas de entrenamiento. TF-IDF se entrenó con una sola semilla. FastText no es reproducible de forma exacta entre plataformas (por ejemplo, 0,807 de accuracy en test en Linux y 0,805 en macOS).
- **Comparación con LLM.** Se probaron dos modelos con un prompt cada uno, con una semilla de ruido y dos intensidades. La latencia de los LLM incluye el servidor y la comunicación HTTP, con cuatro pedidos en paralelo, mientras que la de los demás modelos se mide dentro del mismo proceso, por lo que la comparación solo es válida en orden de magnitud.
- **Sin entrenamiento con ruido.** No se evaluó si entrenar con datos aumentados con ruido permite a los modelos de palabras recuperar robustez.

6. Conclusiones

Se construyeron y compararon cinco clasificadores de intenciones en español sobre MASSIVE, y se midió su degradación frente a siete tipos de ruido de escritura con intensidad controlada y con intervalos de confianza.

La hipótesis se **confirma para los errores de tipeo en los modelos de n-gramas de caracteres**. El control experimental de FastText, en el que solo cambia el uso de subpalabras, muestra una diferencia de caída de 0,448 puntos de accuracy a intensidad máxima, y TF-IDF de caracteres es el modelo más robusto en todas las intensidades de typos y de ruido combinado.

La hipótesis **no se cumple de forma general para todos los tokenizadores de subpalabras**. BETO es el mejor modelo sin ruido, pero ante typos intensos cae casi tanto como los modelos de palabras completas. Su vocabulario WordPiece fijo hace que la palabra mal escrita casi nunca comparta tokens con la original, mientras que los n-gramas de caracteres casi siempre comparten alguno. Lo que parece importar no es solo usar subpalabras, sino cuánto se solapan las representaciones de la forma correcta y la errónea.

Respecto de la pregunta complementaria, la diferencia entre representaciones se manifiesta casi exclusivamente en los errores de tipeo y en el ruido combinado que los incluye. Ante cambios de vocabulario y voseo las caídas son pequeñas, y allí el mejor es BETO, posiblemente por su preentrenamiento. Quitar tildes y abreviar tiene poco efecto sobre todos los modelos.

En la práctica, para un asistente que recibe texto informal, un modelo liviano como TF-IDF de caracteres ofrece el mejor equilibrio entre robustez a typos y costo, y un transformer como BETO convendría combinarlo con corrección ortográfica o con entrenamiento sobre datos con ruido. Los LLM locales evaluados no aportaron mayor robustez que los n-gramas de caracteres y resultaron unas mil veces más lentos.

Como trabajo futuro queda entrenar con datos aumentados con ruido, evaluar con errores reales de usuarios y probar otros transformers, por ejemplo multilingües o con tokenización a nivel de caracteres.

Bibliografía

Bender, Emily M., Timnit Gebru, Angelina McMillan-Major y Shmargaret Shmitchell (2021). «On the Dangers of Stochastic Parrots: Can Language Models Be Too Big?» En: *Proceedings of the 2021 ACM Conference on Fairness, Accountability, and Transparency*, págs. 610-623.

Brown, Tom B., Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal et al. (2020). «Language Models are Few-Shot Learners». En: *Advances in Neural Information Processing Systems*. Vol. 33, págs. 1877-1901.

Devlin, Jacob, Ming-Wei Chang, Kenton Lee y Kristina Toutanova (2019). «BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding». En: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, págs. 4171-4186.

Jurafsky, Daniel y James H. Martin (2024). *Speech and Language Processing*. 3.ª ed. Borrador, agosto de 2024.

Kaplan, Jared, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child et al. (2020). *Scaling Laws for Neural Language Models*. arXiv: 2001.08361.

Li, Qian et al. (2022). «A Survey on Text Classification: From Traditional to Deep Learning». En: *ACM Transactions on Intelligent Systems and Technology* 13.2, 32, págs. 1-41.

Liu, Alisa, Jonathan Hayase, Valentin Hofmann, Sewoong Oh, Noah A. Smith y Yejin Choi (2025). *SuperBPE: Space Travel for Language Models*. arXiv. Versión 1.

Mikolov, Tomas, Kai Chen, Greg Corrado y Jeffrey Dean (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv: 1301.3781.

Qiu, Xipeng, Tianxiang Sun, Yige Xu, Yunfan Shao, Ning Dai y Xuanjing Huang (2021). *Recent Advances in Natural Language Processing via Large Pre-Trained Language Models: A Survey*. arXiv: 2111.01243.

Radford, Alec, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei e Ilya Sutskever (2019). *Language Models are Unsupervised Multitask Learners*. Informe técnico. OpenAI.

Tunstall, Lewis, Leandro von Werra y Thomas Wolf (2022). *Natural Language Processing with Transformers*. O'Reilly Media.

Turney, Peter D. y Patrick Pantel (2010). «From Frequency to Meaning: Vector Space Models of Semantics». En: *Journal of Artificial Intelligence Research* 37, págs. 141-188.

Vaswani, Ashish et al. (2017). «Attention Is All You Need». En: *Advances in Neural Information Processing Systems*. Vol. 30.